Off-Policy GRPO for Online Speculative Decoding

MSAI-635 Interim Report

Gaurav Prakash UID: 122250130
Sanjana Kommareddy UID: 122099364
Simran Kharbanda UID: 122283671

1 Introduction and Scope

Speculative decoding lets a small *draft* model propose K tokens that a large *target* model verifies in one forward pass; verification keeps the longest correct prefix plus one target token, so the output is exactly the target's. The speed-up depends almost entirely on the mean acceptance length $\bar{\tau}$ (accepted draft tokens per verification step). Our proposal argued, following Aurora [1], that drafter training should be *online*, and proposed replacing Aurora's forward-KL distillation loss with an off-policy GRPO objective that rewards accepted length directly.

Change of plan since the proposal. The full Aurora stack needs about six 80 GB GPUs for Qwen3-8B (SGLang at TP=4 plus two FSDP training GPUs) and is CUDA-only (SGLang, Mooncake, Ray, FSDP2, FlexAttention). We first need a working forward-KL baseline that we fully control, so the project has two phases:

1. **Base-paper reproduction (nearly complete):** Aurora's online EAGLE-3 training, scaled down to Qwen3-0.6B on Apple-silicon laptops (PyTorch MPS backend). We reuse Aurora's model, loss and data code and replace only the serving and orchestration infrastructure, in nine sub-tasks (BP-01 to BP-09).
2. **GRPO extension (next):** the off-policy GRPO objective from the proposal, built on the reproduction. The reproduction already logs the quantities GRPO needs: per-round accepted length (the reward) and, for every round, the served draft version and the trainer's version (the staleness Δ).

2 Progress So Far

Seven of the nine base-paper sub-tasks are complete and merged (Table 1); each has a concrete "done when" criterion. The online system (BP-06) and its evaluation (BP-07) now run end to end.

Data (BP-02). Aurora's mixed-domain training data is private, so we built public replacements: 1000 conversations each from ShareGPT (chat), CodeAlpaca-20k (code) and GSM8K (math), filtered to start with a user turn and to fit in 1024 tokens. From these pools we emit three request streams: *mixed* (3000 requests, shuffled), *ordered* (3000, grouped by domain) and *domain-shift* (1200: 600 chat, then 300 code, then 300 math). The draft predicts a reduced 24K-token vocabulary (out of Qwen3's 152K), built from assistant-token frequencies on the mixed stream with Aurora's `generate_vocab_mapping`.

Models (BP-03). The target is Qwen3-0.6B (596M parameters), with auxiliary hidden states captured from layers 1, 13 and 24. The EAGLE-3 draft is a single Llama decoder layer (hidden size 1024) fed by a linear fusion of the three auxiliary states (3072→1024) and the token embedding: 42.4M trainable parameters (24.6M in its own 24K-row output head) plus a frozen 155.6M-parameter embedding copied from the target. Training uses Aurora's `Eagle3Model` with a three-step test-time-training (TTT) unroll and loss $\sum_i 0.8^i \mathcal{L}_i$, where $\mathcal{L}_i$ is the cross-entropy against the target's soft

Table 1: Base-paper sub-tasks and their status.

ID	Work	Done-when criterion	Status
BP-01	Mac environment, device layer (CUDA → MPS → CPU), Aurora installed without its CUDA-only dependencies	Aurora's draft and EAGLE-3 modules import; its parser and vocab tests pass	Done
BP-02	Public datasets (chat, code, math), three request streams, 24K draft-vocab mapping	Streams and cached mapping on disk	Done
BP-03	Qwen3-0.6B target with auxiliary hidden-state capture; EAGLE-3 draft and trainer step on MPS	Draft overfits one batch in 200 steps; time and memory measured	Done
BP-04	Offline static EAGLE-3 baseline trained on held-in chat data	$\bar{\tau}$ clearly > 0 on held-out chat	Done
BP-05	Mini speculative-decoding server: draft, verify, accept, KV cache, logging, training samples, weight hot-swap	Greedy output token-identical to `generate` on 50 prompts	Done
BP-06	Online loop: sample pool, trainer, versioned weight publish; sync and async; day-0 and with-draft starts	$\bar{\tau}$ rises over 300 day-0 requests	Done
BP-07	Metrics, latency bench, CSV/summary writer, plots, online losslessness regression test	Plots and tables from the run logs	Done
BP-08	Experiments E1–E4 (Section 5)	Results table and figures	Next
BP-09	Optional CUDA scale check (Qwen3-1.7B/4B)	Same code runs on CUDA	Optional

distribution at unroll step i.

Static baseline (BP-04). No public EAGLE-3 draft exists for Qwen3-0.6B, so we trained one offline on the chat pool, holding out its last 100 conversations: AdamW at 10^{-4}, warmup plus cosine, effective batch 8, fp32 weights, gradient checkpointing, 8 epochs (896 steps, 19 minutes on MPS). The checkpoint is published as a GitHub release (`bp04-static-v1`) so that every member starts from identical weights. It is both the *static* baseline and the initialisation for Aurora's *with-draft* mode.

Inference server (BP-05). This replaces SGLang with an HF-transformers loop at batch size 1. Each round the draft proposes $K = 3$ guesses and the target scores them in one pass using a `DynamicCache` (cropped on rejection), with greedy (exact argmax match) or speculative-sampling ($\min(1, p/q)$, residual resampling) acceptance. It logs per-round accepted lengths and draft versions, emits Aurora-format training samples, and can load new draft weights between requests.

Online loop (BP-06). The server's training samples go into a bounded *sample pool* (128 samples, kept on the CPU). As in Aurora, serving blocks when the pool is full rather than dropping samples. A separate *online trainer* owns its own fp32 copy of the draft and runs the BP-04 recipe on batches from the pool at a constant learning rate (10^{-4} day-0, 10^{-5} with-draft). Every N steps it *publishes* a copy of its weights into a single pending slot, versioned by its step count; the server applies it before its next request. In *sync* mode the loop alternates between serving B requests and training M steps. In *async* mode a serving thread and a training thread run concurrently; because the MPS backend aborts when two threads issue GPU work at once, a FIFO device lock serialises their GPU calls. Every request records the trainer's version, so per-round staleness Δ is exact.

Evaluation tooling (BP-07). All metrics are recomputed from a run's request log and training log: rolling $\bar{\tau}$, per-position acceptance α_i, acceptance rate, tokens/s against target-only decoding, the estimated speed-up $(1 + \bar{\tau})/(1 + Kc)$, staleness and the loss curve. A latency bench measures c, the cost of one draft step relative to one target step. Output is CSV, JSON and plots. The repository has 68 tests, including losslessness under live weight swaps in sync and async modes.

3 Notional Architecture

Figure 1 shows the system as built. The GRPO extension changes only the trainer's loss and adds tree drafting to the drafter, to produce groups of rollouts; everything else is reused.

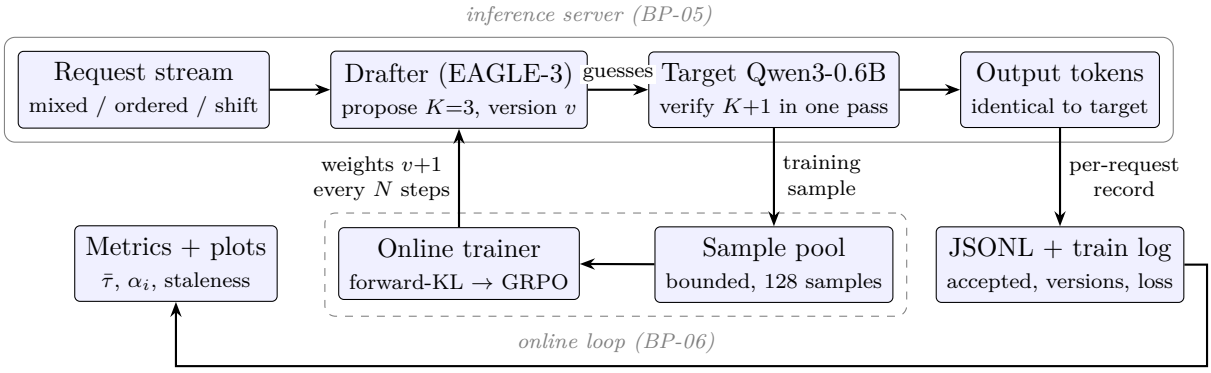


Figure 1: System architecture (BP-02 to BP-07, all implemented). The trainer's objective is forward KL in the reproduction and off-policy GRPO in the extension.

Table 2 maps each Aurora component to its laptop-scale replacement. The algorithmic pieces (draft architecture, TTT loss, vocab mapping, data parsing) are Aurora's own code, imported unchanged.

Table 2: Aurora components and our laptop-scale replacements.

Aurora (paper)	This reproduction
SGLang server with EAGLE-3 patch	HF `transformers` draft–verify loop on MPS
Mooncake transfer store	In-process bounded sample pool with the same backpressure
Ray controller, placement groups	Single loop (sync) or two threads with a device lock (async)
FSDP2 `Eagle3Trainer`	Single-device trainer around Aurora's `Eagle3Model`
FlexAttention / FlashAttention, `torch.compile`	PyTorch SDPA, eager mode
Shared-filesystem weight sync	In-memory versioned publish slot
Qwen3-8B / 4B target, 32K draft vocab	Qwen3-0.6B target, 24K draft vocab
$K = 5$, TTT length 5, ∼20K requests per run	$K = 3$, TTT length 3, 300–3K requests per run

4 Frameworks and Libraries

- **PyTorch 2.14** on the MPS backend (Apple GPU), eager mode with SDPA attention; tests also run with CPU fallback disabled. Python threads for the async loop.
- **Hugging Face** `transformers` 4.57.1 (target model, `DynamicCache`, chat templates), `datasets` and `huggingface_hub` (ShareGPT, CodeAlpaca, GSM8K), `accelerate`, `safetensors`.
- **Aurora** [1] (installed without its CUDA-only dependencies): EAGLE-3 draft model, `Eagle3Model` TTT wrapper, target-feature hooks, vocab mapping, conversation parser.
- **Tooling:** `uv` with Python 3.12, `pytest`, `matplotlib`, a Docker CPU image, Kaggle T4 notebooks for an independent GPU check, GitHub releases for checkpoints and one issue / PR per sub-task.
- **Optional (BP-09):** Aurora's CUDA stack (SGLang, FSDP2, FlexAttention) on a cloud GPU with Qwen3-1.7B/4B.

5 Baselines and State of the Art

Lineage. Speculative decoding with lossless verification was introduced by Leviathan et al. [2] and Chen et al. [3]. Medusa [4] added parallel decoding heads to the target. The EAGLE family [5, 6, 7] drafts from the target's hidden states. EAGLE-3 fuses low-, mid- and high-level features and trains with a test-time-training unroll; it is the drafter used by Aurora and by us. Online Speculative Decoding [8] first proposed distilling the draft on live traffic. Aurora [1] makes this practical as a unified training–serving system with asynchronous weight hot-swap, and is the state of the art we reproduce. GRPO [9] is the critic-free RL objective our extension adapts.

Systems we compare. All are evaluated on the same streams, hardware and decoding settings: (1) **target only**, the speed-up reference; (2) **static EAGLE-3**, the frozen BP-04 draft, i.e. the "train offline, then deploy" pipeline Aurora argues against; (3) **Aurora day-0**, online forward-KL training from a random draft; (4) **Aurora with-draft**, online forward-KL training from the static draft; (5) **ours**, off-policy GRPO with a forward-KL anchor, with the ablations from the proposal (synchronous on-policy GRPO, $\Delta = 0$; group size G; KL weight β; clipping and truncation thresholds; reward shaping).

Planned experiments (BP-08). **E1**: day-0 online training against target-only on the mixed stream (partly done: Section 6). **E2**: static draft vs. with-draft online training under the chat→code→math shift; Aurora's main claim is faster recovery after a shift, which we measure as the area under the rolling-$\bar{\tau}$ curve after each shift. **E3**: E2 on the ordered stream. **E4**: sensitivity to the sync interval $N \in \{5, 20, 50\}$, the learning rate, and sync vs. async training. The headline metric is $\bar{\tau}$, which does not depend on hardware; we also report α_i, measured and estimated speed-up, and the staleness distribution.

6 Evaluation Results (Prototype)

All runs use greedy decoding, $K = 3$, fp32 target and draft, and up to 128 new tokens. BP-03 to BP-05 ran on an M5 Pro (24 GB); the BP-06 run ran on a MacBook Air (16 GB). Tokens/s is therefore only compared within a run.

Correctness. Speculative output is token-identical to target-only decoding on every request checked: 50/50 stream prompts and 100/100 held-out chat requests with a fixed draft (BP-05), and **300/300** requests of the online run while the draft's weights were replaced 30 times (BP-06).

Draft trainability and static baseline (BP-03, BP-04). On one fixed batch the draft overfits in 200 steps (loss 9.87→0.851 against a target-entropy floor of 0.849; accuracy 0.99 / 0.99 / 0.98). The static draft's held-out accuracy (draft argmax equals target argmax, teacher-forced) rises from 0.183 after epoch 1 to 0.245 after epoch 2 and **0.310 / 0.270 / 0.248** after epoch 8, plateauing after epoch 6. A 2-epoch run gives the same numbers on a Kaggle T4 and on MPS, and the released T4 8-epoch checkpoint matches the MPS one ($\bar{\tau}$ 0.315 vs. 0.313).

Static draft in the server (BP-05). Table 3 gives end-to-end results on the M5 Pro. The upper rows are sanity bounds: an untrained draft accepts nothing, while a draft overfitted to the exact answers reaches $\bar{\tau} = 2.6$ and twice the target-only speed, so the server can deliver a real speed-up when the draft guesses well. On held-out chat the trained static draft reaches $\bar{\tau} = 0.313$.

Online day-0 run (BP-06, BP-07). We served the first 300 requests of the mixed stream (104 chat, 100 code, 96 math) starting from a *random* draft, in sync mode: one optimizer step per 2 requests (each sample used once), learning rate 10^{-4}, a publish every 5 steps. That is 150 training steps and 30 weight swaps; the run took 55 minutes, 2.3 of which were training, with a peak of 7.5 GB. Figure 2 and Table 4 show the result: rolling $\bar{\tau}$ rises from **0.039** over the first 50 requests

Table 3: Speculative serving with a fixed draft (M5 Pro). α_i is the share of rounds whose first i guesses were all accepted, so $\bar{\tau} = \alpha_1 + \alpha_2 + \alpha_3$.

Draft	Requests	$\bar{\tau}$	$\alpha_1/\alpha_2/\alpha_3$	tok/s (spec. vs. target)	Lossless
Untrained, KV cache	50 stream	0.00	–	33.4 vs. 59.9	50/50
Overfit (upper bound)	50 stream	2.60	–	122.3 vs. 59.9 (2.0×)	50/50
Static, 2 epochs	100 held-out	0.181	0.164 / 0.016 / 0.001	29.1 vs. 54.9 (0.53×)	100/100
Static, 8 epochs (baseline)	100 held-out	**0.313**	**0.242 / 0.058 / 0.013**	**32.1 vs. 54.8 (0.59×)**	**100/100**

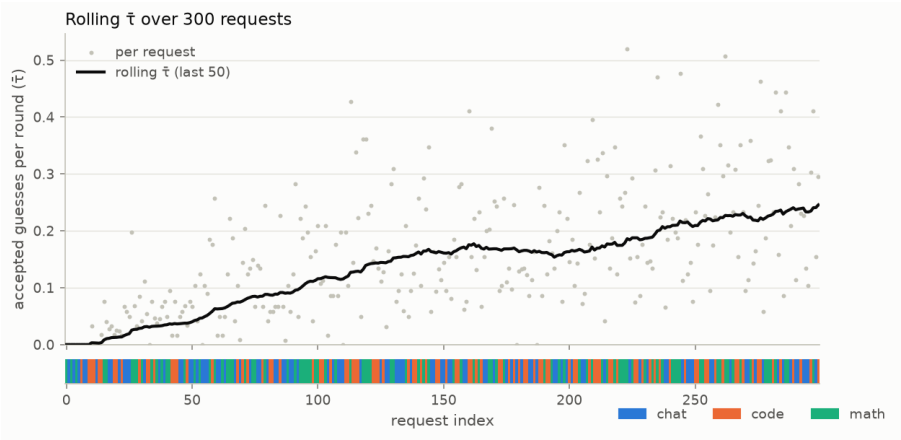


Figure 2: Day-0 online run: per-request $\bar{\tau}$ (dots) and rolling $\bar{\tau}$ over the last 50 requests, with each request's domain underneath.

to **0.246** over the last 50, and is still rising at request 300. That meets BP-06's done-when.

A with-draft smoke run (20 mixed requests, starting from the released static draft, learning rate 10^{-5}) starts at $\bar{\tau} \approx 0.26$ immediately, which the day-0 draft only reaches after about 300 requests. A full with-draft run is part of E2.

What the numbers say.

1. **Online learning works from scratch.** With only 150 steps on about 34K completion tokens, the day-0 draft reaches 80% of the static draft's $\bar{\tau}$, which needed 896 offline steps. It learns fastest on math, whose text is the most predictable (math 0.115 → 0.318, code 0.074 → 0.226, chat 0.050 → 0.126, first vs. last third of each domain's requests).
2. **Acceptance is almost all at the first guess.** α_2 and α_3 stay small for both drafts. Guesses 2–3 are built on the draft's own hidden states, and a chain stops at the first miss, so errors compound. Teacher-forced accuracy (0.31 / 0.27 / 0.25) therefore overstates live acceptance. Positions 2–3 are exactly where an acceptance-based reward differs most from forward KL, which weights all positions equally.
3. **Staleness is small and harmless.** The served draft lags the trainer by 0–4 steps (mean 1.96), and losslessness held through every swap.
4. **Still slower than the target alone.** A round costs about 2.2–2.3 target-only steps (68 ms on the Air, 41 ms on the M5 Pro) but yields only $1 + \bar{\tau}$ tokens, so break-even needs $\bar{\tau} \approx 1.2$–1.3. The estimate (0.89×) is more optimistic than the measurement (0.53×) because it leaves out the $(K+1)$-token verify pass, the draft re-reading a growing context (it has no KV cache yet) and per-round overhead.

Relation to published results. Papers usually report acceptance length as tokens per target pass, which is $1 + \bar{\tau}$ in our notation. Published EAGLE-2/3 drafts with tree drafting on 8B–70B

Table 4: Day-0 online run, 300 mixed requests (MacBook Air). Measured speed is relative to target-only decoding of the same requests; the estimate uses the bench's $c = 0.099$.

	Requests	$\bar{\tau}$	$\alpha_1/\alpha_2/\alpha_3$	tok/s (vs. target)	Est. speed-up	Δ mean / max	Lossless
chat	104	0.090	0.087 / 0.003 / 0.000	13.6 (0.45×)	0.84×	2.08 / 4	104/104
code	100	0.144	0.137 / 0.006 / 0.000	19.1 (0.58×)	0.88×	1.92 / 4	100/100
math	96	0.227	0.204 / 0.019 / 0.003	19.7 (0.61×)	0.95×	1.89 / 4	96/96
all	300	0.152	0.141 / 0.009 / 0.001	16.9 (0.53×)	0.89×	1.96 / 4	300/300
first 50	50	0.039	0.038 / 0.000 / 0.000	–	–	–	–
last 50	50	**0.246**	**0.215 / 0.028 / 0.003**	–	0.96×	–	–

targets report roughly 4–6+ tokens per pass and 3–6× speed-ups; chain-style methods such as classic speculative decoding and Medusa report about 2–3× [2, 4, 6, 7]. We are at 1.25–1.31 tokens per pass and below 1×. The gap is expected: our drafts have seen thousands rather than hundreds of thousands of training conversations, we draft one chain of $K = 3$ instead of a tree, a 0.6B target makes the draft's relative cost large ($c \approx 0.1$), and MPS with batch size 1 lacks the kernels and batching of production servers. We therefore compare systems *against each other* on the same setup, with $\bar{\tau}$ as the headline metric, rather than against published absolute speed-ups.

7 Issues Faced

Solved.

- **Compute and CUDA-only stack:** scaled down to Qwen3-0.6B on MPS, replacing SGLang, Mooncake, Ray and FSDP2 (Table 2) while reusing Aurora's algorithmic code unchanged.
- **No public draft, private data:** trained our own EAGLE-3 draft on public data; ShareGPT filters drop 939 threads that start mid-conversation and 613 longer than 1024 tokens.
- **Draft-vocab coverage:** 16K covered only 97.6% of assistant tokens (others can never be drafted); 24K covers 0.993 / 0.995 / 0.998 (chat / code / math) for 8.2M more parameters.
- **Memory:** 4×1K-token batches needed 9.9 GB (draft logits × 3 TTT steps); batch 2 with gradient checkpointing trains in 2.4 GB, and the whole online loop peaks at 7.5 GB, so it fits a 16 GB laptop.
- **Precision:** pure-bf16 AdamW lost 2–3 accuracy points (small updates round away), so we train in fp32 like Aurora's master weights. In bf16, a 4-token verify pass rounds differently from 4 single steps and can flip near-ties, so losslessness checks run in fp32 (450/450 pass).
- **Threads on MPS:** MPS aborts when two threads use the GPU at once; async mode serialises GPU calls with a FIFO device lock.
- **Sharing checkpoints:** the 170 MB static draft is a GitHub release with a SHA-256 checksum.

Open.

- **No wall-clock speed-up yet.** Levers: higher $\bar{\tau}$ (more online training, GRPO), a draft KV cache, and tree drafting.
- **Train/test overlap in the streams.** The 600 chat requests of the domain-shift stream, and 900 of the 1000 chat requests in the mixed stream, were BP-04 training prompts. This does not affect day-0 runs, but before E2/E3 we must rebuild the chat segments from held-out conversations or report on unseen requests only.
- **Single-device asynchrony.** With the device lock, "async" on one GPU interleaves rather than overlaps serving and training. Staleness is real but smaller than in Aurora's two-server layout; E4 will quantify the difference.
- **Hardware variance.** A fanless laptop slowed ∼12% over an hour (the speed ratio held at 0.53–0.54), so we only compare speeds within a run.

- **GRPO prerequisites.** Groups of rollouts need tree drafting (top-$k > 1$), which is not yet implemented, and $K = 3$ gives a shorter reward range than the paper's $K = 5$.

8 Next Steps

1. **With-draft run:** 300 mixed requests from the released static draft, with the same settings as the day-0 run, for a direct comparison.
2. **BP-08, experiments E1–E4**, after removing the stream overlap: E2/E3 on the domain-shift and ordered streams (static vs. day-0 vs. with-draft), and E4 sweeps over N, learning rate and sync vs. async.
3. **GRPO extension:** tree drafting for groups; reward $R(a) = L(a) - \lambda\kappa(K - L(a))$ from the logged accepted lengths; the off-policy objective with truncated importance weights and the forward-KL anchor, using the logged staleness; and comparison against systems 2–4 above.
4. **BP-09 (optional):** the same code on a CUDA GPU with a larger target, to check that the trends carry over.

References

[1] J. Wang, F. Bie, J. Li, Z. Zhou, Z. Shao, Y. Wang, Y. Liu, Q. Wu, A. May, S. Yanamandra, Y. Zhang, C. Zhang, T. Dao, P. Liang, B. Athiwaratkun, S. L. Song, C. Xu, and X. Wu. When RL meets adaptive speculative training: A unified training–serving system. *arXiv:2602.06932*, 2026.

[2] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[3] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. *arXiv:2302.01318*, 2023.

[4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *ICML*, 2024.

[5] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *ICML*, 2024.

[6] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *EMNLP*, 2024.

[7] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv:2503.01840*, 2025.

[8] X. Liu, L. Hu, P. Bailis, A. Cheung, Z. Deng, I. Stoica, and H. Zhang. Online speculative decoding. In *ICML*, 2024.

[9] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024.